

```

"""MCP Server Evaluation Harness

This script evaluates MCP servers by running test questions against them using Claude.
"""

import argparse
import asyncio
import json
import re
import sys
import time
import traceback
import xml.etree.ElementTree as ET
from pathlib import Path
from typing import Any

from anthropic import Anthropic

from connections import create_connection

EVALUATION_PROMPT = """You are an AI assistant with access to tools.

When given a task, you MUST:
1. Use the available tools to complete the task
2. Provide summary of each step in your approach, wrapped in <summary> tags
3. Provide feedback on the tools provided, wrapped in <feedback> tags
4. Provide your final response, wrapped in <response> tags

Summary Requirements:
- In your <summary> tags, you must explain:
  - The steps you took to complete the task
  - Which tools you used, in what order, and why
  - The inputs you provided to each tool
  - The outputs you received from each tool
  - A summary for how you arrived at the response

Feedback Requirements:
- In your <feedback> tags, provide constructive feedback on the tools:
  - Comment on tool names: Are they clear and descriptive?
  - Comment on input parameters: Are they well-documented? Are required vs optional
parameters clear?
  - Comment on descriptions: Do they accurately describe what the tool does?
  - Comment on any errors encountered during tool usage: Did the tool fail to execute?
Did the tool return too many tokens?
  - Identify specific areas for improvement and explain WHY they would help
  - Be specific and actionable in your suggestions

Response Requirements:
- Your response should be concise and directly address what was asked
- Always wrap your final response in <response> tags
- If you cannot solve the task return <response>NOT_FOUND</response>
- For numeric responses, provide just the number
- For IDs, provide just the ID
- For names or text, provide the exact text requested
- Your response should go last"""

```

```

def parse_evaluation_file(file_path: Path) -> list[dict[str, Any]]:
    """Parse XML evaluation file with qa_pair elements."""
    try:
        tree = ET.parse(file_path)
        root = tree.getroot()
        evaluations = []

        for qa_pair in root.findall("./qa_pair"):
            question_elem = qa_pair.find("question")
            answer_elem = qa_pair.find("answer")

            if question_elem is not None and answer_elem is not None:
                evaluations.append({
                    "question": (question_elem.text or "").strip(),
                    "answer": (answer_elem.text or "").strip(),
                })

        return evaluations
    except Exception as e:
        print(f"Error parsing evaluation file {file_path}: {e}")
        return []

def extract_xml_content(text: str, tag: str) -> str | None:
    """Extract content from XML tags."""
    pattern = rf"<{tag}>(.*?)</{tag}>"
    matches = re.findall(pattern, text, re.DOTALL)
    return matches[-1].strip() if matches else None

async def agent_loop(
    client: Anthropic,
    model: str,
    question: str,
    tools: list[dict[str, Any]],
    connection: Any,
) -> tuple[str, dict[str, Any]]:
    """Run the agent loop with MCP tools."""
    messages = [{"role": "user", "content": question}]

    response = await asyncio.to_thread(
        client.messages.create,
        model=model,
        max_tokens=4096,
        system=EVALUATION_PROMPT,
        messages=messages,
        tools=tools,
    )

    messages.append({"role": "assistant", "content": response.content})

    tool_metrics = {}

```

```

while response.stop_reason == "tool_use":
    tool_use = next(block for block in response.content if block.type ==
"tool_use")
    tool_name = tool_use.name
    tool_input = tool_use.input

    tool_start_ts = time.time()
    try:
        tool_result = await connection.call_tool(tool_name, tool_input)
        tool_response = json.dumps(tool_result) if isinstance(tool_result, (dict,
list)) else str(tool_result)
    except Exception as e:
        tool_response = f"Error executing tool {tool_name}: {str(e)}\n"
        tool_response += traceback.format_exc()
    tool_duration = time.time() - tool_start_ts

    if tool_name not in tool_metrics:
        tool_metrics[tool_name] = {"count": 0, "durations": []}
    tool_metrics[tool_name]["count"] += 1
    tool_metrics[tool_name]["durations"].append(tool_duration)

    messages.append({
        "role": "user",
        "content": [{
            "type": "tool_result",
            "tool_use_id": tool_use.id,
            "content": tool_response,
        }]
    })

    response = await asyncio.to_thread(
        client.messages.create,
        model=model,
        max_tokens=4096,
        system=EVALUATION_PROMPT,
        messages=messages,
        tools=tools,
    )
    messages.append({"role": "assistant", "content": response.content})

    response_text = next(
        (block.text for block in response.content if hasattr(block, "text")),
        None,
    )
    return response_text, tool_metrics

async def evaluate_single_task(
    client: Anthropic,
    model: str,
    qa_pair: dict[str, Any],
    tools: list[dict[str, Any]],
    connection: Any,
    task_index: int,
) -> dict[str, Any]:

```

```

    """Evaluate a single QA pair with the given tools."""
    start_time = time.time()

    print(f"Task {task_index + 1}: Running task with question: {qa_pair['question']}")
    response, tool_metrics = await agent_loop(client, model, qa_pair["question"],
tools, connection)

    response_value = extract_xml_content(response, "response")
    summary = extract_xml_content(response, "summary")
    feedback = extract_xml_content(response, "feedback")

    duration_seconds = time.time() - start_time

    return {
        "question": qa_pair["question"],
        "expected": qa_pair["answer"],
        "actual": response_value,
        "score": int(response_value == qa_pair["answer"]) if response_value else 0,
        "total_duration": duration_seconds,
        "tool_calls": tool_metrics,
        "num_tool_calls": sum(len(metrics["durations"]) for metrics in
tool_metrics.values()),
        "summary": summary,
        "feedback": feedback,
    }

REPORT_HEADER = """
# Evaluation Report

## Summary

- **Accuracy**: {correct}/{total} ({accuracy:.1f}%)
- **Average Task Duration**: {average_duration_s:.2f}s
- **Average Tool Calls per Task**: {average_tool_calls:.2f}
- **Total Tool Calls**: {total_tool_calls}

---
"""

TASK_TEMPLATE = """
### Task {task_num}

**Question**: {question}
**Ground Truth Answer**: `{expected_answer}`
**Actual Answer**: `{actual_answer}`
**Correct**: {correct_indicator}
**Duration**: {total_duration:.2f}s
**Tool Calls**: {tool_calls}

**Summary**
{summary}

**Feedback**
{feedback}

```

```

---
"""

async def run_evaluation(
    eval_path: Path,
    connection: Any,
    model: str = "claude-3-7-sonnet-20250219",
) -> str:
    """Run evaluation with MCP server tools."""
    print("🚀 Starting Evaluation")

    client = Anthropic()

    tools = await connection.list_tools()
    print(f"📋 Loaded {len(tools)} tools from MCP server")

    qa_pairs = parse_evaluation_file(eval_path)
    print(f"📋 Loaded {len(qa_pairs)} evaluation tasks")

    results = []
    for i, qa_pair in enumerate(qa_pairs):
        print(f"Processing task {i + 1}/{len(qa_pairs)}")
        result = await evaluate_single_task(client, model, qa_pair, tools, connection,
i)
        results.append(result)

    correct = sum(r["score"] for r in results)
    accuracy = (correct / len(results)) * 100 if results else 0
    average_duration_s = sum(r["total_duration"] for r in results) / len(results) if
results else 0
    average_tool_calls = sum(r["num_tool_calls"] for r in results) / len(results) if
results else 0
    total_tool_calls = sum(r["num_tool_calls"] for r in results)

    report = REPORT_HEADER.format(
        correct=correct,
        total=len(results),
        accuracy=accuracy,
        average_duration_s=average_duration_s,
        average_tool_calls=average_tool_calls,
        total_tool_calls=total_tool_calls,
    )

    report += ".join([
        TASK_TEMPLATE.format(
            task_num=i + 1,
            question=qa_pair["question"],
            expected_answer=qa_pair["answer"],
            actual_answer=result["actual"] or "N/A",
            correct_indicator="✅" if result["score"] else "❌",
            total_duration=result["total_duration"],
            tool_calls=json.dumps(result["tool_calls"], indent=2),
            summary=result["summary"] or "N/A",

```

```

        feedback=result["feedback"] or "N/A",
    )
    for i, (qa_pair, result) in enumerate(zip(qa_pairs, results))
])

return report

def parse_headers(header_list: list[str]) -> dict[str, str]:
    """Parse header strings in format 'Key: Value' into a dictionary."""
    headers = {}
    if not header_list:
        return headers

    for header in header_list:
        if ":" in header:
            key, value = header.split(":", 1)
            headers[key.strip()] = value.strip()
        else:
            print(f"Warning: Ignoring malformed header: {header}")
    return headers

def parse_env_vars(env_list: list[str]) -> dict[str, str]:
    """Parse environment variable strings in format 'KEY=VALUE' into a dictionary."""
    env = {}
    if not env_list:
        return env

    for env_var in env_list:
        if "=" in env_var:
            key, value = env_var.split("=", 1)
            env[key.strip()] = value.strip()
        else:
            print(f"Warning: Ignoring malformed environment variable: {env_var}")
    return env

async def main():
    parser = argparse.ArgumentParser(
        description="Evaluate MCP servers using test questions",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Examples:
# Evaluate a local stdio MCP server
python evaluation.py -t stdio -c python -a my_server.py eval.xml

# Evaluate an SSE MCP server
python evaluation.py -t sse -u https://example.com/mcp -H "Authorization: Bearer
token" eval.xml

# Evaluate an HTTP MCP server with custom model
python evaluation.py -t http -u https://example.com/mcp -m claude-3-5-sonnet-20241022
eval.xml
"""
    ),

```

```

)

parser.add_argument("eval_file", type=Path, help="Path to evaluation XML file")
parser.add_argument("-t", "--transport", choices=["stdio", "sse", "http"],
default="stdio", help="Transport type (default: stdio)")
parser.add_argument("-m", "--model", default="claude-3-7-sonnet-20250219",
help="Claude model to use (default: claude-3-7-sonnet-20250219)")

stdio_group = parser.add_argument_group("stdio options")
stdio_group.add_argument("-c", "--command", help="Command to run MCP server (stdio
only)")
stdio_group.add_argument("-a", "--args", nargs="+", help="Arguments for the command
(stdio only)")
stdio_group.add_argument("-e", "--env", nargs="+", help="Environment variables in
KEY=VALUE format (stdio only)")

remote_group = parser.add_argument_group("sse/http options")
remote_group.add_argument("-u", "--url", help="MCP server URL (sse/http only)")
remote_group.add_argument("-H", "--header", nargs="+", dest="headers", help="HTTP
headers in 'Key: Value' format (sse/http only)")

parser.add_argument("-o", "--output", type=Path, help="Output file for evaluation
report (default: stdout)")

args = parser.parse_args()

if not args.eval_file.exists():
    print(f"Error: Evaluation file not found: {args.eval_file}")
    sys.exit(1)

headers = parse_headers(args.headers) if args.headers else None
env_vars = parse_env_vars(args.env) if args.env else None

try:
    connection = create_connection(
        transport=args.transport,
        command=args.command,
        args=args.args,
        env=env_vars,
        url=args.url,
        headers=headers,
    )
except ValueError as e:
    print(f"Error: {e}")
    sys.exit(1)

print(f"🔗 Connecting to MCP server via {args.transport}...")

async with connection:
    print("✅ Connected successfully")
    report = await run_evaluation(args.eval_file, connection, args.model)

    if args.output:
        args.output.write_text(report)
        print(f"\n✅ Report saved to {args.output}")

```

```
        else:
            print("\n" + report)

if __name__ == "__main__":
    asyncio.run(main())
```